Warehouse Ops

Exception & Resolution Workbench

INSY 4325 | Team Project Final Report

Team 5 | Spring 2026

Abdullah Moghal | David Chacon | Mir Ali | Rahat Hossain | Rayden Siarot

React / Vite	Spring Boot	Supabase	Vercel / Railway

Live: ops-workbench-team-5.vercel.app

GitHub: github.com/AbdullahMoghal/ops-workbench-team-5

Table of Contents

1 Abstract

Warehouse Ops: Exception & Resolution Workbench is a full-stack web application that centralizes the lifecycle of warehouse inventory exceptions, from issue reporting through supervisor review, discrepancy resolution, and audit tracking. The solution integrates a React/Vite frontend, a Spring Boot REST backend, and Supabase (PostgreSQL + Auth) to provide role-based workflows, operational visibility dashboards, and a durable audit trail. The project improves decision speed, accountability, and data consistency by replacing fragmented and manual exception handling with a unified digital process.

4.2 hrs	4 Roles	10 Tables	3 Services
Avg Resolution Time	User Roles	DB Tables	Core Services

2 Company Background

This project is designed for a warehouse operations context where inventory accuracy, process compliance, and response times directly affect fulfillment quality and cost. In a typical mid-size warehouse operation, associates report discrepancies (missing, damaged, or misrouted SKUs), supervisors review and assign actions, operations managers monitor trends, and administrators manage users and master data. As volume grows, email/spreadsheet and manual log processes become error-prone and slow, motivating the need for an integrated exception-resolution platform.

The deployed product operates across three cloud services: Vercel (React frontend), Railway (Spring Boot backend), and Supabase (PostgreSQL and JWT auth), forming a production-ready three-tier architecture suitable for real warehouse SME deployment.

3 Problem Domain

3.1 Context & Environment

Warehouse teams handle high-frequency inventory events across locations, bins, categories, and SKUs. Exception handling must occur in near-real time with clear ownership, escalation paths, and historical traceability. The system targets warehouse environments with multiple physical zones (e.g., A-A-12-04) organized by warehouse, zone, and bin identifiers.

3.2 Stakeholders & Roles

Role	Responsibilities
Warehouse Associate	Reports exceptions, views own ticket workflows.
Supervisor	Reviews queue, assigns tickets, approves or rejects adjustments.
Ops Manager	Monitors performance metrics and exception trends across all tickets.

Admin	Manages users, roles, and all master or reference data.

3.3 Current Issues / Problems

- Fragmented reporting channels and poor handoffs between shifts and roles.
- Delayed ticket assignment and resolution due to lack of a central queue.
- Inconsistent prioritization and status updates leading to missed SLAs.
- Weak auditability for approvals and changes with no immutable log.
- Limited visibility into recurring root-cause patterns across exception types.

3.4 Scope & Constraints

In Scope	Out of Scope
Ticket and issue reporting and tracking	ERP / WMS deep integrations
Review queue and assignment workflows	Automated ML recommendations in production
Discrepancy logging and adjustment decisions	File evidence storage and upload
Dashboard, reports, and audit trail	Notification automation (future work)
User and role administration, master data CRUD	Advanced analytics export pipelines

Constraints

- Must support role-based access control enforced at both API and UI layers.
- Must run with cloud deployment: Vercel, Railway, and Supabase.
- Must preserve data integrity and support auditable, immutable action logs.
- Must operate with practical latency suitable for daily warehouse operations.

3.5 Clear Objectives

- Reduce time from issue report to resolution by centralizing the ticket lifecycle.
- Improve consistency of workflow and approval decisions through enforced role rules.
- Increase visibility into exception volume, types, and status via live dashboards.
- Ensure complete, queryable audit history of all key workflow actions.

4 Functional Requirements

The following functional requirements (FRs) define the complete scope of system behavior. Each is atomic, testable, and traceable to UI screens and use cases.

ID	Requirement	Acceptance Criteria	Priority
FR-01	Authentication & Session: The system shall authenticate users and establish role context.	Valid credentials create authenticated session; invalid credentials return actionable error.	P1
FR-02	RBAC: The system shall enforce role permissions across API and UI.	Unauthorized role receives denial; authorized role can execute action.	P1
FR-03	Ticket Creation: Users may create tickets with location, item, type, description, and value impact.	Ticket persists with unique ticket number and initial status.	P1
FR-04	Review Queue: Filterable ticket list by status, priority, and date.	Filters return matching set; no filters returns full queue.	P1
FR-05	Ticket Assignment: Authorized users assign tickets and trigger status transitions.	Assignment and status updates persist and appear in UI and history.	P1
FR-06	Discrepancy Logging: Record expected vs. actual quantity and variance value.	One discrepancy per ticket; duplicate rejected with error.	P1
FR-07	Adjustment Decision: Supervisors/Ops/Admin approve or reject adjustments with rationale.	Decision updates ticket outcome and creates audit event.	P1
FR-08	Audit Trail: Record immutable action logs with actor, action, details, and timestamp.	Logs retrievable globally and per ticket in descending time order.	P1
FR-09	Dashboard & Reports: Provide summary metrics and pattern or status visualizations.	KPIs and charts reflect current persisted ticket data.	P2
FR-10	Admin Master Data: Admin manages users, roles, locations, categories, and inventory items.	CRUD operations validated and persisted with constraints enforced.	P2

Business Rules

- Priority is auto-derived from ticket type and estimated value impact.
- Closed tickets cannot be modified or have new discrepancies logged.
- Only one discrepancy record is permitted per ticket.
- Only authorized roles (Supervisor, Ops Manager, Admin) can approve or reject adjustments.
- All major workflow actions automatically generate immutable audit log entries.

5 UI Design

The application uses a warm beige theme (#F5F0E8 background) with orange accent (#E8530A) and dark near-black topbar, directly matching the brand identity. A fixed sidebar and topbar provide predictable navigation. Reusable components (status badges, data tables, KPI cards, filter bars) ensure visual and functional consistency across all screens.

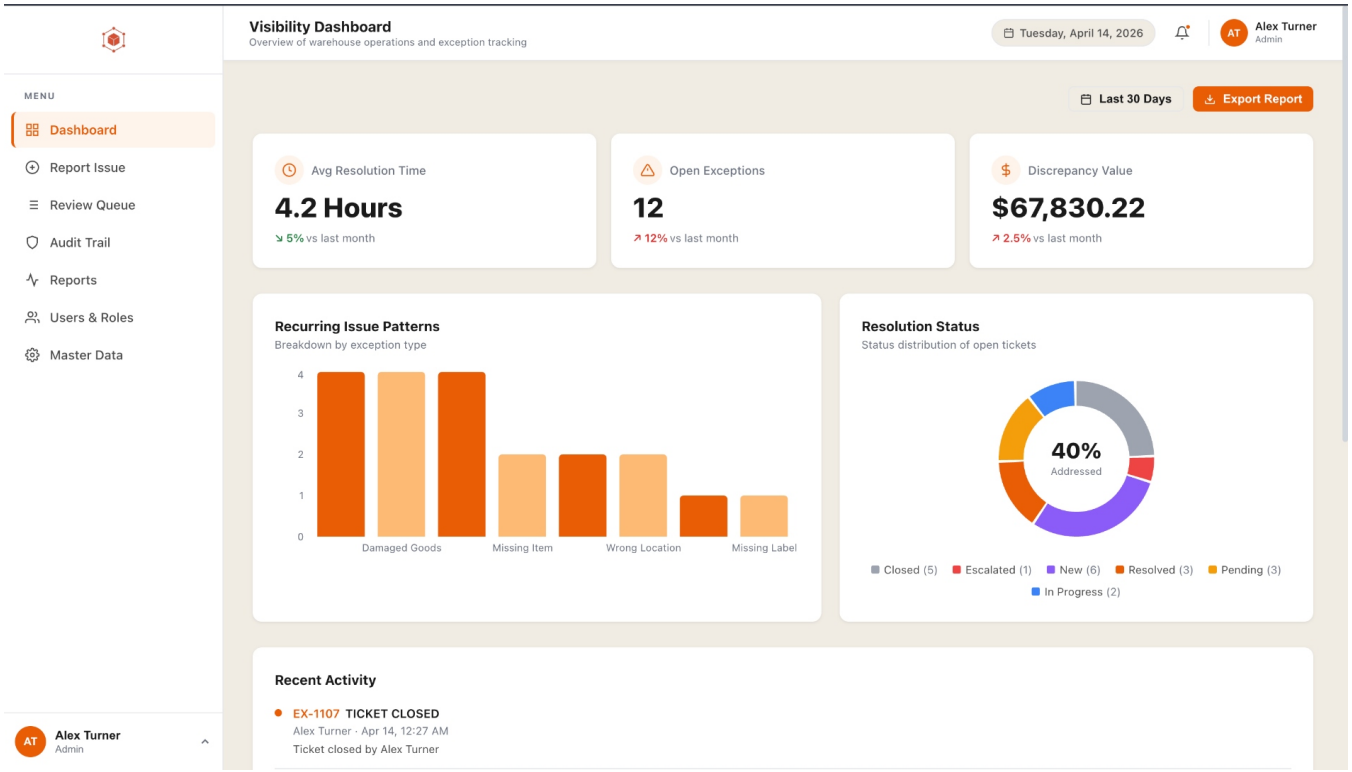


Figure 1 – Visibility Dashboard: KPI cards, recurring issue patterns, and resolution status.

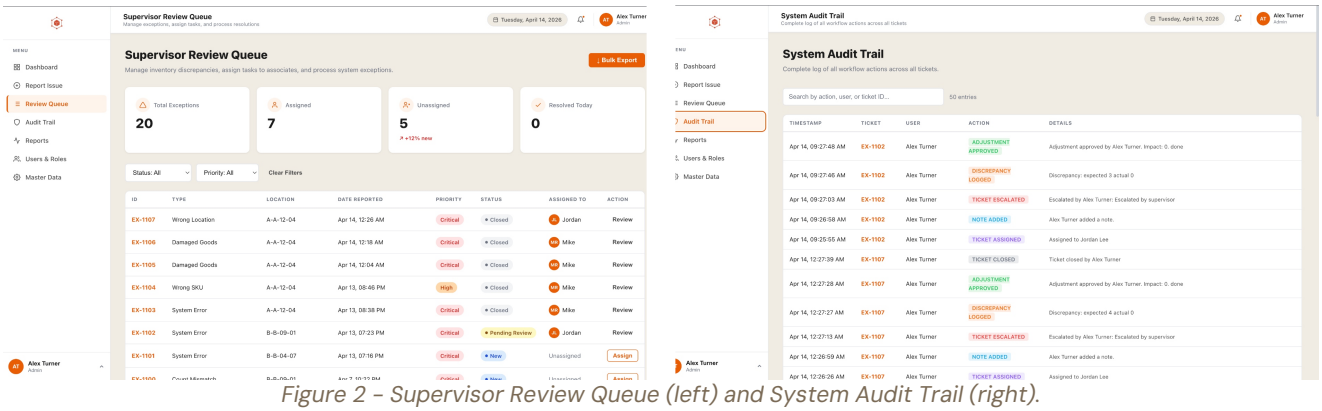


Figure 2 – Supervisor Review Queue (left) and System Audit Trail (right).

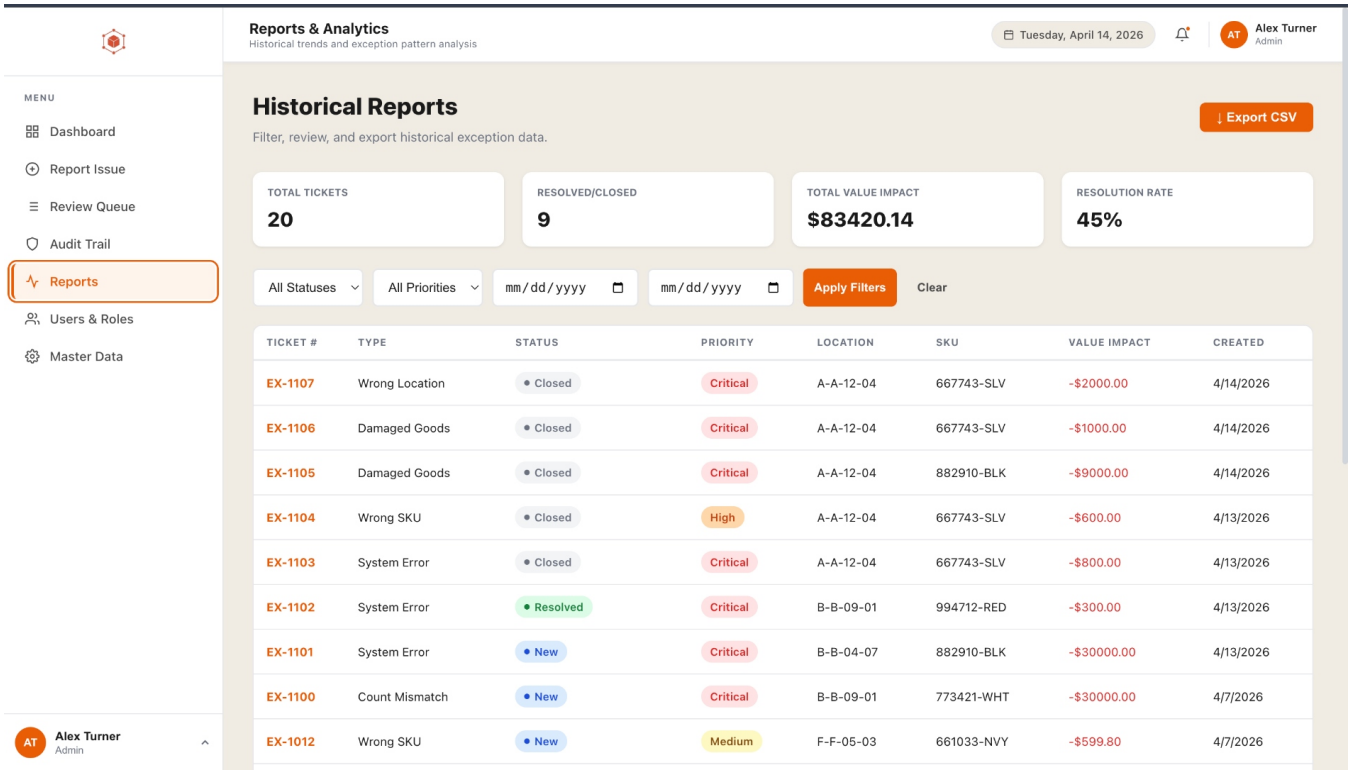


Figure 3 – Reports & Analytics: historical exception table with priority, status, value impact, and CSV export.

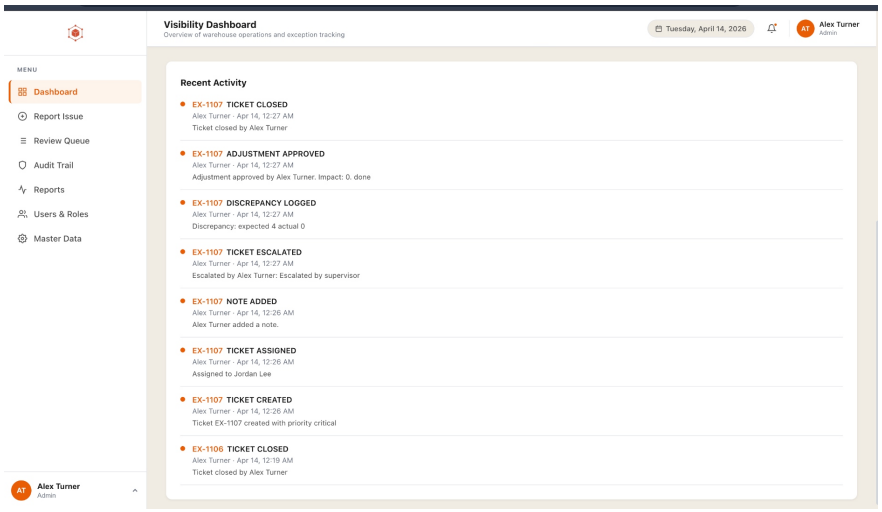
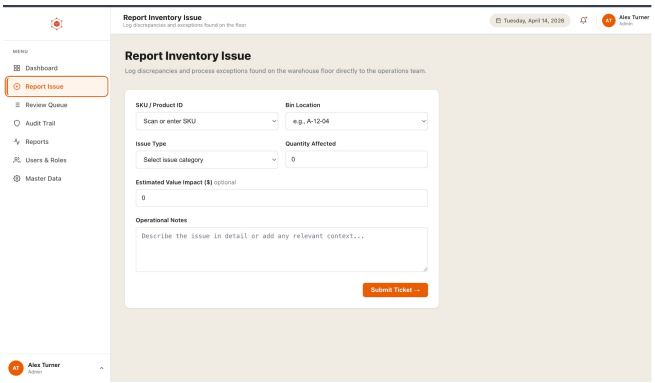


Figure 4 – Dashboard Recent Activity feed showing ticket lifecycle events with timestamps.



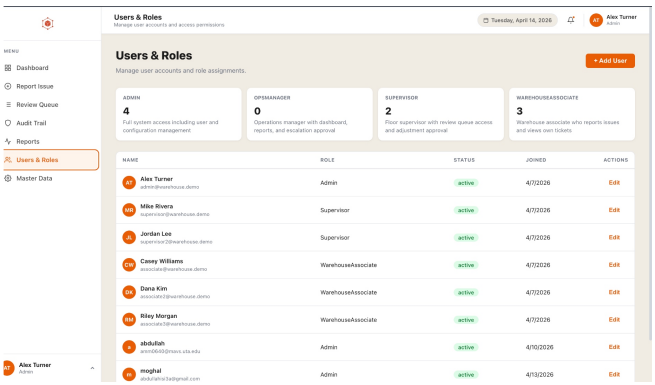


Figure 5 – Report Issue form.

Figure 6 – Admin Account Settings.

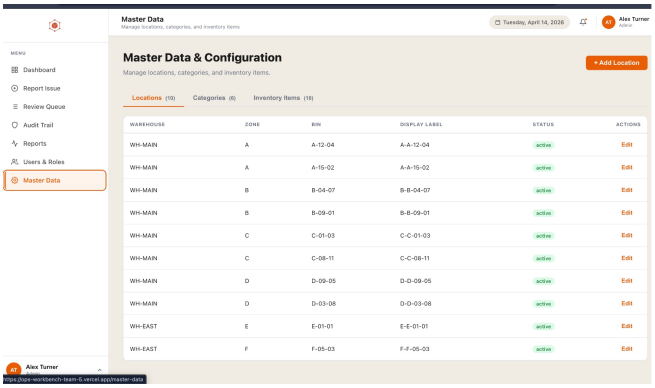


Figure 7 – Admin Master Data Configuration.

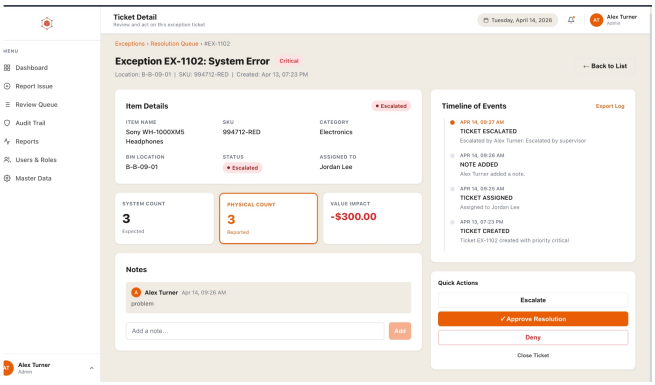


Figure 8 – Ticket Detail in Review Queue.

UI Design Principles

- Core screens present: Login, Dashboard, Report Issue, Review Queue, Ticket Detail, Audit Trail, Reports, Users & Roles, Master Data.
- Reusable components: status/priority badges, filterable tables, KPI cards, sidebar navigation, and topbar.
- Warm beige theme (#F5F0E8) with orange accent (#E8530A) and dark topbar for clear hierarchy.
- States and validation: empty states (no records), error states (API or auth failures), and success states (created, updated, approved).
- Usability basics: labeled form fields, required indicators, contrast-focused badge styling, and predictable navigation.
- UI maps directly to role workflows and functional requirements: report, review, resolve, audit, report.

6 Class Diagram

The UML class diagram below models the core domain entities, their attributes, service layer classes, and all associations and dependencies. Service classes (TicketService, DiscrepancyService, AuditLogService) are shown with their full method signatures to illustrate behavioral coverage.

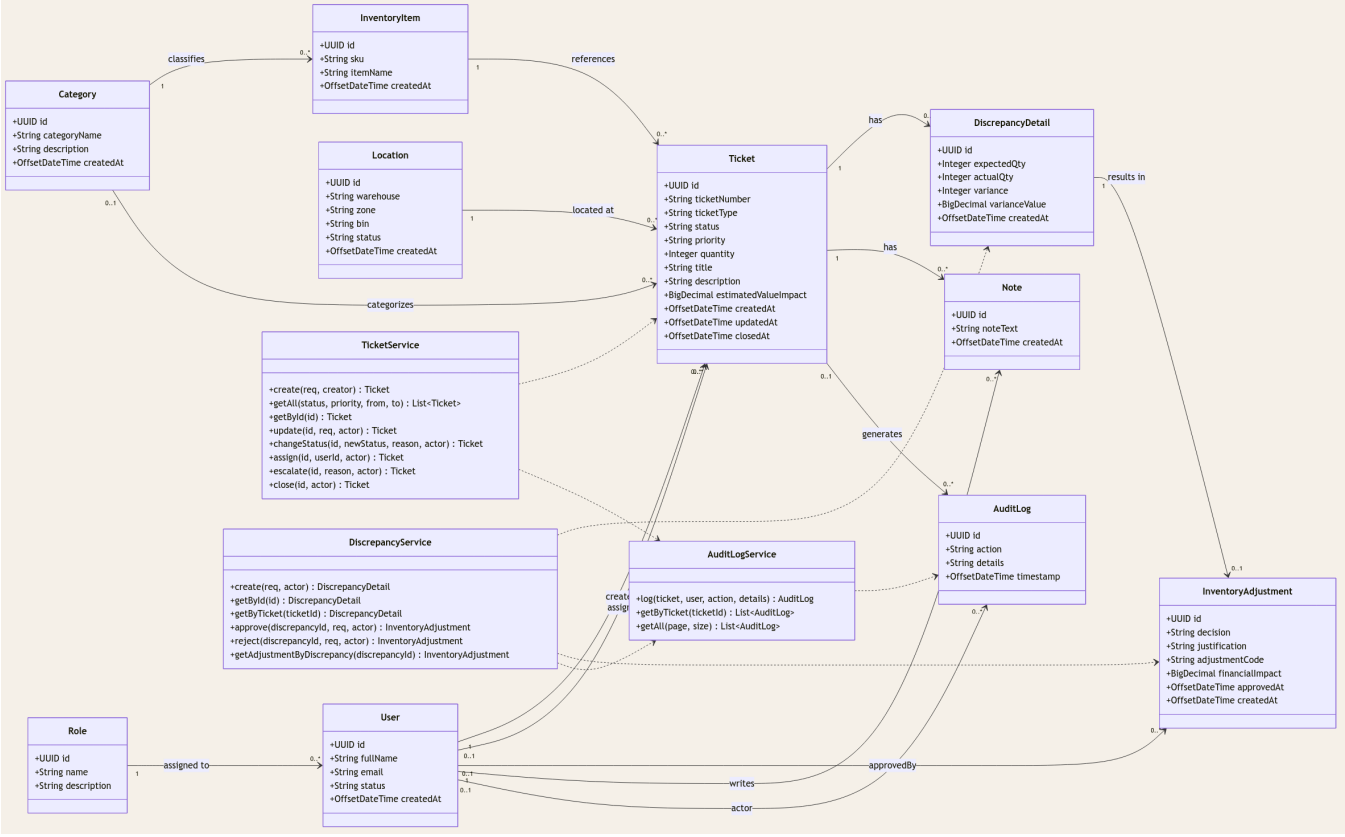


Figure 9 – UML Class Diagram with domain entities, service classes, and associations.

Key Design Decisions

- Classes & Attributes: All domain entities (Role, User, Location, Category, InventoryItem, Ticket, DiscrepancyDetail, InventoryAdjustment, Note, AuditLog) carry UUID PKs, typed fields, and OffsetDateTime timestamps.
- Relationships: Associations modeled with direction arrows (e.g., User creates Ticket, Ticket has DiscrepancyDetail, DiscrepancyDetail results in InventoryAdjustment).
- Multiplicities: One Role to many Users; one Ticket to zero-or-one DiscrepancyDetail (enforcing one-discrepancy-per-ticket rule); one DiscrepancyDetail to zero-or-one InventoryAdjustment.
- Methods / Behavior: TicketService exposes full CRUD plus changeStatus, assign, escalate, and close. DiscrepancyService exposes approve and reject. AuditLogService exposes log and retrieval.
- Dependencies: Service classes depend on (dashed arrow) their primary entity and share a dependency on AuditLogService, ensuring all workflow actions produce audit records.

7 Database Diagram (ERD)

The entity-relationship diagram below documents the full Supabase PostgreSQL schema. All tables use UUID primary keys and explicit foreign key constraints. The schema is normalized to third normal form (3NF), with separate lookup, transactional, and log entity groups.

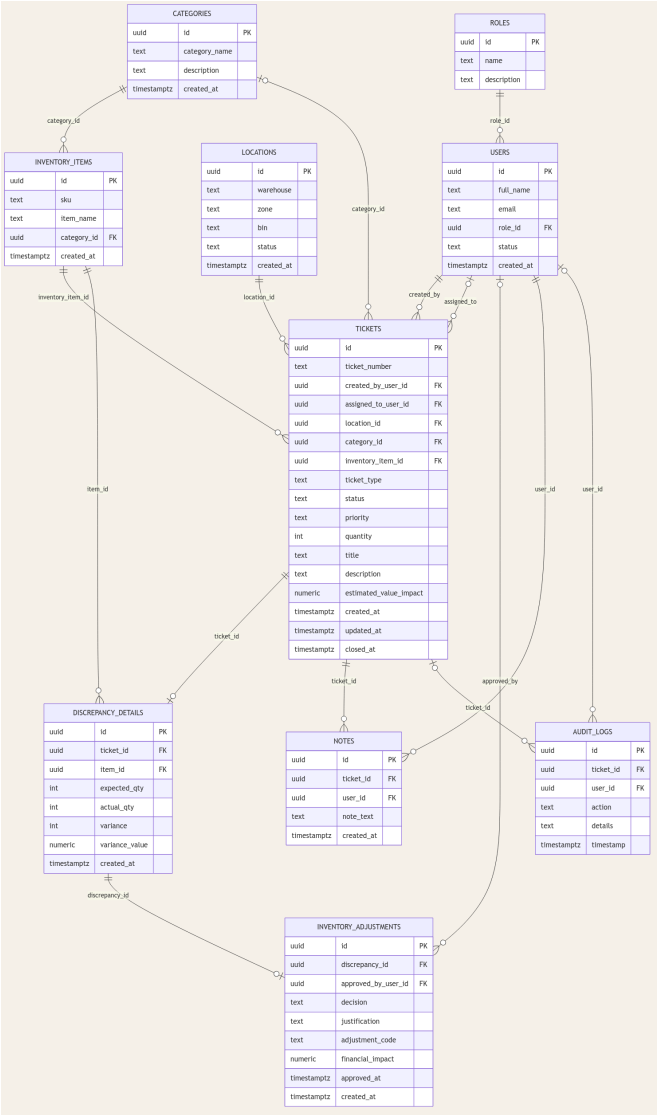


Figure 10 – Database ERD: all 10 tables with PK/FK relationships and cardinalities.

Schema Notes

- Tables & Columns: 10 tables covering roles, users, locations, categories, inventory_items, tickets, discrepancy_details, inventory_adjustments, notes, and audit_logs.
- Primary Keys: UUID PKs on all tables, ensuring globally unique identifiers compatible with Supabase's distributed architecture.
- Foreign Keys & Relationships: roles.id referenced by users.role_id; categories.id referenced by inventory_items and tickets; tickets.id referenced by discrepancy_details, notes, and audit_logs.

- Unique Constraints: email, sku, category_name, ticket_number, and composite (warehouse, zone, bin) on locations.
- One-to-one constraints: discrepancy_details.ticket_id (UK) and inventory_adjustments.discrepancy_id (UK) enforce the single-discrepancy and single-adjustment-per-discrepancy business rules.
- Normalization: Schema is 3NF - lookup entities (roles, categories, locations), transactional entities (tickets, discrepancies, adjustments), and dependent log entities (notes, audit_logs) are fully separated.
- Check Constraints: Domain value enforcement on status, priority, and ticket_type columns prevents invalid state entries.

8 Actual Implementation

Frontend

The React/Vite single-page application is organized into route-level pages and a shared component library. Axios handles all REST API calls, and the Supabase JS client manages authentication state and JWT session persistence. Dashboard charts render live exception metrics using recharts.

- React 18 + Vite with route-level code splitting for performance.
- Axios API layer with request interceptors for JWT Authorization headers.
- Supabase client for auth, session refresh, and logout.
- Recharts for bar and donut visualizations on the Dashboard screen.
- Reusable badge, table, form, card, and layout components.
- Hosted on Vercel with environment variables: VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY, VITE_API_URL.

Backend

The Spring Boot REST API applies a layered architecture: controllers handle HTTP routing, services encapsulate business rules, and JPA repositories map to the Supabase PostgreSQL schema. Spring Security with a custom JWT filter validates every request against the Supabase-issued JWT secret and extracts role claims for authorization decisions.

- Spring Boot 3 with layered controller / service / repository architecture.
- Spring Security 6 + custom JWT filter for stateless role-based authorization.
- JPA (Hibernate) entities mapped to Supabase PostgreSQL tables via connection pooler.
- Global exception handler returning structured error responses.
- @Transactional service methods ensuring atomicity on multi-step workflows.
- Deployed on Railway reading: SUPABASE_DB_URL, SUPABASE_DB_USER, SUPABASE_DB_PASSWORD, SUPABASE_JWT_SECRET, CORS_ORIGINS.

Data / Auth / Infrastructure

Layer	Technology	Role
Database	Supabase PostgreSQL (pooler)	Persistent storage for all 10 tables via PgBouncer connection pooler.
Authentication	Supabase Auth + JWT	Issues JWTs on login; backend validates signature and extracts role claims.
Frontend Hosting	Vercel (static)	Serves React SPA as static files with CDN edge distribution.
Backend Hosting	Railway (container)	Runs Spring Boot JAR with runtime environment variables for secrets.

9 Conclusions & Recommendations

The platform successfully meets core warehouse exception-management objectives: centralized workflow, role-based controls, traceable actions, and operational visibility. Architecture choices are practical and deployment-ready for academic and SME use. The three-tier cloud stack (Vercel, Railway, Supabase) demonstrates sound separation of concerns and is cost-effective for small to mid-scale warehouse operations.

Recommended Next Steps

- Production Hardening: Implement JWT token refresh strategy to prevent session expiry in long operations.
- Row-Level Security (RLS): Adopt Supabase RLS policies as an additional authorization layer beyond Spring Security.
- Stricter Auth Validation: Add audience (aud) and issuer (iss) claim validation on the JWT filter.
- Usability Refinements: Add file attachment support for evidence photos and inline ticket comment threading.
- Advanced Analytics: Expand the Reports screen with trend lines, drill-down filtering, and PDF/Excel export.
- Notification Workflows: Implement email or in-app alerts for critical ticket escalations and SLA breaches.
- Test Coverage: Expand unit and integration test suites for service layer and API endpoint edge cases.

10 High-Level Implementation Plan

Phase	Activities
1. Foundation	Define schema and entity relationships; write and run Supabase migrations.
2. Core APIs	Build ticket, discrepancy, adjustment, and audit log REST endpoints in Spring Boot.
3. RBAC / Auth	Integrate Supabase JWT-based authentication; implement Spring Security role filter.
4. Frontend Flows	Implement Login, Dashboard, Report Issue, Review Queue, and Ticket Detail screens.
5. Admin / Reports	Add Users/Roles/Master Data administration screens and Reports/Analytics page.
6. Deployment	Configure Vercel (frontend) and Railway (backend) with Supabase environment variables.
7. Stabilization	Fix CORS and JWT configuration; resolve connection pooler settings; validate end-to-end.
8. Documentation	Finalize README, UML/ERD diagrams, and this final project report.

11 Summary of Project

Warehouse Ops is a complete full-stack implementation of exception and resolution workflow management for warehouse operations. It supports role-driven actions, business-rule enforcement, discrepancy and adjustment handling, and complete auditability across the entire ticket lifecycle. The system is deployed across three modern cloud services and demonstrates practical software engineering from requirements capture through database design, backend API construction, frontend delivery, and cloud operations.

Capability	Status
Role-based authentication and authorization	Implemented
Ticket creation, assignment, escalation, and closure	Implemented
Discrepancy logging with expected vs. actual variance	Implemented
Supervisor approval / rejection workflow	Implemented
Immutable audit trail per ticket and system-wide	Implemented
KPI dashboard with live metrics and charts	Implemented
Historical reports with filters and CSV export	Implemented
Admin CRUD for users, roles, locations, categories, items	Implemented
File attachments and notification workflows	Future Work

12 References

[1] Supabase Documentation (Auth, PostgreSQL, Connection Pooler). https://supabase.com/docs

[2] Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/docs/current/reference/html/

[3] Spring Security Reference. https://docs.spring.io/spring-security/reference/

[4] Vercel Documentation. https://vercel.com/docs

[5] Railway Documentation. https://docs.railway.com

[6] Mermaid Diagramming (UML and ERD rendering). https://mermaid.js.org

[7] React Documentation. https://react.dev

[8] Vite Build Tool. https://vitejs.dev

[9] Project Repository: ops-workbench-team-5 (GitHub). https://github.com/AbdullahMoghal/ops-workbench-team-5

13 Appendices

A. Environment Variables

Frontend (Vercel):

`VITE_SUPABASE_URL`

`VITE_SUPABASE_ANON_KEY`

`VITE_API_URL`

Backend (Railway):

`SUPABASE_DB_URL`

`SUPABASE_DB_USER`

`SUPABASE_DB_PASSWORD`

`SUPABASE_JWT_SECRET`

`SPRING_PROFILES_ACTIVE`

`CORS_ORIGINS`

B. Test Scenarios

- Login as each role (Associate, Supervisor, Ops Manager, Admin) and verify correct route and action visibility.
- Create ticket as Associate, assign in Review Queue as Supervisor, log discrepancy, approve adjustment, verify audit log entries.
- Filter Review Queue and Reports page with and without status, priority, and date range parameters.
- Validate 403 Forbidden responses for unauthorized role actions (e.g., Associate attempting to approve adjustment).

- Attempt to create a second discrepancy on a ticket with existing discrepancy; verify rejection.
- Attempt to modify a Closed ticket; verify 400 or 422 response.

C. Known Future Enhancements

- File attachments and photographic evidence upload per ticket.
- Notification workflows for critical escalations and SLA deadline alerts.
- Advanced analytics with trend charts, export to PDF and Excel.
- Production-grade session refresh and Supabase Row-Level Security policy hardening.
- Mobile-responsive layout optimization for warehouse floor tablet use.